

Develop the Midas Core System

PROJECT OVERVIEW

The Midas Core System is a simulation of a real-world financial transaction engine. The objective was to design and implement a system capable of processing high-volume user transactions reliably, while ensuring data integrity, database persistence, and external API integration for incentive handling. In this project, I gain hands-on experience with distributed systems, database integration, API communication, and backend validation logic.

TOOLS & TECHNOLOGIES

- **Java (Spring Boot, Spring Data JPA, Spring Kafka, Spring MVC)**
 - Backend service development, event-driven architecture, and REST API design.
- **H2 Database** – a lightweight relational database for persistent transaction and user data.
- **Apache Kafka** – an event streaming platform for transaction ingestion.
- **REST API (Spring RestTemplate, Spring Web)**
 - external service integration with Incentives API and new balance API exposure.
- **JUnit / Debugger** – for unit testing and validation of business logic.
- **Git** – version control for project tracking.

WHAT I DID

Transaction Validation via Kafka

- Implemented a `KafkaListener` to consume incoming transaction messages.
- Designed validation logic to ensure:
 - Both sender and recipient IDs are valid users.
 - Sender has sufficient balance to cover the transaction.
- Ensured invalid transactions are discarded without altering system state.

I Built the foundation for reliable financial operations by preventing fraudulent or invalid transactions.

Database Integration (H2 + JPA)

- Designed a new `TransactionRecord` entity with `@Entity` annotations to persist validated transactions.
- Created a `TransactionRecordRepository` (Spring Data JPA) to handle CRUD operations.
- Implemented many-to-one relationships between transactions and User entities to enable full traceability.
- Updated balances for sender and recipient in a transactional, consistent manner.

Enabled persistent audit trails and accurate financial records through database-backed transaction storage.

External Incentives API Integration

- Integrated the Incentives API running on port 8080 via Spring's `RestTemplate`.
- Posted validated transactions as JSON and parsed returned incentive amounts.
- Extended `TransactionRecord` to include incentive data.
- Enhanced business logic so that incentive amounts are credited only to the recipient without impacting sender balances.

Successfully connected the core system to an external microservice, modeling a real-world incentive/loyalty program.

Balance Query REST API

- Added a new REST Controller (`/balance` endpoint) to Midas Core, running on port 33400.
- Implemented a GET request handler that:
 - Accepts `userId` as a request parameter.
 - Returns a serialized instance of the provided `Balance` class.
 - Returns balance 0 if the requested user does not exist.
- Designed a solution so that it coexists with Kafka listener and incentive integration in the same Spring Boot application.
- Contribution: Extended Midas Core with a direct user-facing API, enabling real-time balance queries and improving system usability, while considering architectural trade-offs between simplicity and modularity.

Key Learnings & Outcomes

- Gained practical experience with event-driven architectures (Kafka).
- Learned to design transaction-safe database operations using JPA and relationships between entities.
- Strengthened skills in external service integration and handling real-world API responses.
- Developed an understanding of core financial system design principles, including validation, persistence, and extensibility.
- Practiced debugging and testing workflows to ensure the correctness of the system state after complex transaction flows.